

03_app-js

#javascript

#frontend

#api

#fastapi

#SIEM

#SOC

1 Ubicación del archivo dentro del proyecto

```
siem-lab/  
├── frontend/  
│   └── assets/  
│       └── app.js
```

El archivo `app.js` se encuentra dentro de:

```
frontend/assets/
```

Este archivo contiene funciones comunes reutilizadas por las páginas del frontend.

No está ligado a una sola pantalla. Actúa como una pequeña librería interna para que `alerts.js` y `alert_detail.js` no tengan que repetir código.

2 Comando utilizado para visualizar el archivo

```
cd ~/siem-lab  
sed -n '1,420p' frontend/assets/app.js
```

Desglose del comando:

```
cd ~/siem-lab
```

Sitúa la terminal en la raíz del proyecto.

```
sed
```

Permite visualizar contenido de archivos.

```
-n
```

Evita imprimir todo automáticamente.

```
'1,420p'
```

Imprime de la línea 1 a la 420.

```
frontend/assets/app.js
```

Ruta del archivo analizado.

3 Código completo del archivo

```
// Configurable: si sirves el frontend desde otro puerto/host, ajusta esto.  
const API_BASE = "http://localhost:8000"; // mismo host/puerto que el backend  
  
function qs(id) { return document.getElementById(id); }  
  
function show(el, msg) {  
  el.textContent = msg;  
  el.classList.remove("hidden");  
}
```

```

function hide(el) {
  el.textContent = "";
  el.classList.add("hidden");
}

function fmtDate(isoOrDate) {
  try {
    const d = new Date(isoOrDate);
    return d.toLocaleString();
  } catch {
    return String(isoOrDate ?? "");
  }
}

function statusBadgeClass(status) {
  // Sin colores específicos; mantenemos estilo neutro con borde.
  return "badge";
}

async function apiFetch(path, { method = "GET", query = null, body = null } = {}) {
  const url = new URL(API_BASE + path, window.location.origin);
  if (query) {
    Object.entries(query).forEach(([k, v]) => {
      if (v === null || v === undefined || v === "") return;
      url.searchParams.set(k, String(v));
    });
  }

  const init = {
    method,
    headers: { "Accept": "application/json" },
  };

  if (body !== null) {
    init.headers["Content-Type"] = "application/json";
    init.body = JSON.stringify(body);
  }

  const res = await fetch(url.toString(), init);
  const text = await res.text();
  let data = null;
  try { data = text ? JSON.parse(text) : null; } catch { /* noop */ }

  if (!res.ok) {
    const detail = data?.detail ?? text ?? `HTTP ${res.status}`;
    throw new Error(detail);
  }
  return data;
}

function getQueryParam(name) {
  const u = new URL(window.location.href);
  return u.searchParams.get(name);
}

function setQueryParams(params) {
  const u = new URL(window.location.href);
  Object.entries(params).forEach(([k, v]) => {
    if (v === null || v === undefined || v === "") u.searchParams.delete(k);
    else u.searchParams.set(k, String(v));
  });
  window.history.replaceState({}, "", u.toString());
}

```

4 Función general del archivo

`app.js` centraliza utilidades comunes del frontend.

Su papel es evitar repetir lógica en varios archivos JavaScript.

Las funciones principales son:

<code>API_BASE</code>	→ define la URL base del backend
<code>qs()</code>	→ busca elementos del DOM por id
<code>show()</code>	→ muestra mensajes
<code>hide()</code>	→ oculta mensajes
<code>fmtDate()</code>	→ formatea fechas
<code>statusBadgeClass()</code>	→ devuelve clase CSS para estados
<code>apiFetch()</code>	→ realiza llamadas HTTP a la API
<code>getQueryParam()</code>	→ lee parámetros de la URL
<code>setQueryParams()</code>	→ actualiza parámetros de la URL

Este archivo es usado por:

```
frontend/assets/alerts.js
frontend/assets/alert_detail.js
```

La relación es:

```
app.js
  ↓
funciones comunes

alerts.js
  ↓
usa apiFetch(), qs(), show(), hide(), fmtDate(), setQueryParams()

alert_detail.js
  ↓
usa apiFetch(), qs(), show(), hide(), fmtDate(), getQueryParam()
```

5 Estructura general del archivo

El archivo puede dividirse en siete bloques:

```
const API_BASE = "http://localhost:8000";
```

Configuración de la URL base de la API.

```
function qs(id) { ... }
```

Función corta para seleccionar elementos del DOM.

```
function show(el, msg) { ... }
function hide(el) { ... }
```

Funciones para mostrar u ocultar mensajes.

```
function fmtDate(isoOrDate) { ... }
```

Función para formatear fechas.

```
function statusBadgeClass(status) { ... }
```

Función para devolver clase visual de estado.

```
async function apiFetch(...) { ... }
```

Función central para llamadas HTTP.

```
function getQueryParam(name) { ... }  
function setQueryParams(params) { ... }
```

Funciones para trabajar con parámetros de URL.

Visualmente:

```
app.js  
├─ API_BASE  
├─ qs()  
├─ show()  
├─ hide()  
├─ fmtDate()  
├─ statusBadgeClass()  
├─ apiFetch()  
├─ getQueryParam()  
└─ setQueryParams()
```

6 Análisis línea por línea

Comentario de configuración

```
// Configurable: si sirves el frontend desde otro puerto/host, ajusta esto.
```

Es un comentario.

Indica que la URL del backend puede cambiar si el frontend o el backend se sirven desde otro host o puerto.

No afecta a la ejecución.

Constante API_BASE

```
const API_BASE = "http://localhost:8000"; // mismo host/puerto que el backend
```

Define la URL base de la API.

Desglose:

```
const
```

Declara una constante.

Una constante no debería reasignarse después.

```
API_BASE
```

Nombre de la constante.

```
"http://localhost:8000"
```

URL donde se espera encontrar el backend FastAPI.

En este proyecto, el backend se expone en:

```
http://localhost:8000
```

Esto coincide con Docker Compose, donde la API publica el puerto 8000.

El comentario final:

```
// mismo host/puerto que el backend
```

recuerda que esta URL apunta al backend.

Ejemplo de uso:

```
apiFetch("/alerts")
```

se convierte en:

```
http://localhost:8000/alerts
```

Función `qs`

```
function qs(id) { return document.getElementById(id); }
```

Esta función simplifica la selección de elementos HTML por `id`.

Desglose:

```
function
```

Palabra clave para definir una función.

```
qs
```

Nombre de la función.

Probablemente viene de “query selector”, aunque realmente usa `getElementById`.

```
id
```

Parámetro recibido.

Representa el ID del elemento HTML.

```
document.getElementById(id)
```

Busca en el documento HTML un elemento con ese ID.

Ejemplo:

```
qs("errorBox")
```

equivale a:

```
document.getElementById("errorBox")
```

Esto hace el código más corto.

Función `show`

```
function show(el, msg) {  
  el.textContent = msg;  
  el.classList.remove("hidden");  
}
```

Esta función muestra un elemento y le asigna un mensaje.

Se usa para mostrar errores o mensajes informativos.

Parámetros de `show`

```
el
```

Elemento HTML que se quiere mostrar.

Ejemplo:

```
qs("errorBox")
```

```
msg
```

Texto que se quiere mostrar dentro del elemento.

Asignar texto

```
el.textContent = msg;
```

Modifica el contenido textual del elemento.

Usar `textContent` es más seguro que usar `innerHTML` cuando solo se quiere mostrar texto, porque no interpreta HTML.

Ejemplo:

```
show(qs("errorBox"), "Error cargando alertas");
```

Resultado visible:

```
Error cargando alertas
```

Mostrar el elemento

```
el.classList.remove("hidden");
```

Elimina la clase CSS `hidden`.

En `styles.css`, esta clase tiene:

```
.hidden { display: none; }
```

Por tanto, al quitar `hidden`, el elemento vuelve a mostrarse.

Función `hide`

```
function hide(el) {  
  el.textContent = "";  
  el.classList.add("hidden");  
}
```

Esta función oculta un elemento y borra su contenido.

Borrar el contenido

```
el.textContent = "";
```

Vacía el texto del elemento.

Esto evita que quede un mensaje antiguo guardado.

Ocultar el elemento

```
el.classList.add("hidden");
```

Añade la clase `hidden`.

Como `hidden` tiene `display: none`, el elemento deja de mostrarse.

Función `fmtDate`

```
function fmtDate(isoOrDate) {  
  try {  
    const d = new Date(isoOrDate);  
    return d.toLocaleString();  
  } catch {  
    return String(isoOrDate ?? "");  
  }  
}
```

Esta función formatea fechas para mostrarlas de forma más legible.

Recibe un valor que puede ser una fecha ISO o un objeto fecha.

Inicio de función

```
function fmtDate(isoOrDate) {
```

Define la función `fmtDate`.

El parámetro:

```
isoOrDate
```

representa la fecha recibida.

Ejemplo:

```
2026-01-15T12:00:00
```

Bloque `try`

```
try {
```

Intenta ejecutar el código que puede fallar.

Aquí se usa porque la conversión de fecha podría recibir un valor inesperado.

Crear objeto `Date`

```
const d = new Date(isoOrDate);
```

Crea un objeto `Date` de JavaScript a partir del valor recibido.

Ejemplo:

```
new Date("2026-01-15T12:00:00")
```

Formatear fecha

```
return d.toLocaleString();
```

Devuelve la fecha en formato local del navegador.

Esto convierte una fecha ISO en un formato más legible para el usuario.

Captura de error

```
} catch {
```

Si algo falla dentro del `try`, se ejecuta el bloque `catch`.

Aquí no se captura una variable de error concreta, porque no se necesita.

Retorno alternativo

```
return String(isoOrDate ?? "");
```

Si la fecha no se puede formatear, devuelve el valor original convertido a texto.

El operador:

```
??
```

es el operador de fusión nula.

Significa:

```
si isoOrDate es null o undefined, usar ""
```

Por tanto:

```
String(isoOrDate ?? "")
```

evita devolver `null` o `undefined` directamente.

Función `statusBadgeClass`

```
function statusBadgeClass(status) {  
  // Sin colores específicos; mantenemos estilo neutro con borde.  
  return "badge";  
}
```

Esta función devuelve la clase CSS que se aplicará a un estado de alerta.

En esta versión devuelve siempre:

```
badge
```

Parámetro `status`


```
status
```

Representa el estado de la alerta:

```
open  
ack  
closed
```

Aunque actualmente no se usa para cambiar el estilo, queda preparado para futuras mejoras.

Comentario interno

```
// Sin colores específicos; mantenemos estilo neutro con borde.
```

Explica que no se asignan colores diferentes según el estado.

Todos los estados usan el mismo estilo visual.

Retorno

```
return "badge";
```

Devuelve la clase CSS `badge`.

Esta clase se define en `styles.css`.

Ejemplo de uso en `alerts.js`:

```
<span class="${statusBadgeClass(a.status)}">${a.status}</span>
```

Y en `alert_detail.js`:

```
qs("v_status").className = statusBadgeClass(a.status);
```

Función `apiFetch`

```
async function apiFetch(path, { method = "GET", query = null, body = null } = {}) {
```

Esta es la función más importante de `app.js`.

Centraliza las llamadas HTTP al backend.

Permite hacer peticiones GET, PATCH u otros métodos, enviar query params y enviar body JSON.

`async`

```
async
```

Indica que la función es asíncrona.

Dentro se puede usar:

```
await
```

Esto es necesario porque `fetch()` trabaja con promesas.

Nombre de función

```
apiFetch
```

Nombre de la función.

Indica que es una función para llamar a la API.

Parámetro `path`

```
path
```

Ruta relativa del endpoint.

Ejemplos:

```
/alerts  
/alerts/1  
/alerts/1/ui
```

La función la combinará con `API_BASE`.

Parámetro de configuración

```
{ method = "GET", query = null, body = null } = {}
```

Este parámetro usa destructuring con valores por defecto.

Permite llamar a la función de varias formas:

```
apiFetch("/alerts")
```

o:

```
apiFetch("/alerts/1", {  
  method: "PATCH",  
  body: { status: "ack" }  
})
```

Valores por defecto:

```
method = "GET"  
query = null  
body = null
```

El `<empty object>` final permite llamar a `apiFetch("/alerts")` sin pasar segundo argumento.

Construcción de URL

```
const url = new URL(API_BASE + path, window.location.origin);
```

Crea un objeto `URL`.

Desglose:

```
API_BASE + path
```

Une la URL base con la ruta.

Ejemplo:

```
http://localhost:8000 + /alerts
```

Resultado:

```
http://localhost:8000/alerts
```

```
window.location.origin
```

Se usa como base de referencia.

En este caso, al usar una URL absoluta en `API_BASE`, el segundo argumento no tiene mucho peso, pero ayuda a que `new URL()` tenga una base válida.

Comprobación de query params

```
if (query) {
```

Si se han pasado parámetros de consulta, se procesan.

Ejemplo de query:

```
{
  limit: 50,
  offset: 0,
  status: "open",
  group_key: "server-01"
}
```

Recorrer parámetros

```
Object.entries(query).forEach(([k, v]) => {
```

Convierte el objeto `query` en pares clave-valor.

Ejemplo:

```
Object.entries({ limit: 50, status: "open" })
```

produce:

```
[["limit", 50], ["status", "open"]]
```

El destructuring:

```
([k, v])
```

separa cada par en:

```
k → clave
v → valor
```

Ignorar valores vacíos

```
if (v === null || v === undefined || v === "") return;
```

Si el valor es `null`, `undefined` o cadena vacía, no se añade a la URL.

Esto evita construir URLs como:

```
/alerts?status=&group_key=
```

Solo se envían filtros reales.

Añadir query param

```
url.searchParams.set(k, String(v));
```

Añade el parámetro a la URL.

Ejemplo:

```
url.searchParams.set("status", "open")
```

Resultado:

```
/alerts?status=open
```

`String(v)` convierte el valor a texto.

Objeto `init`

```
const init = {  
  method,  
  headers: { "Accept": "application/json" },  
};
```

Define la configuración que se pasará a `fetch()`.

Método HTTP

```
method
```

Puede ser:

```
GET  
POST  
PATCH  
DELETE
```

En este proyecto se usa sobre todo:

```
GET  
PATCH
```

Header `Accept`

```
headers: { "Accept": "application/json" }
```

Indica al backend que el frontend espera recibir JSON.

Comprobación de `body`

```
if (body !== null) {
```

Si se ha pasado un cuerpo de petición, se prepara para enviarlo como JSON.

Esto ocurre, por ejemplo, al actualizar una alerta:

```
body: { status: "ack" }
```

Header Content-Type

```
init.headers["Content-Type"] = "application/json";
```

Indica que el cuerpo enviado será JSON.

Esto es necesario para que FastAPI interprete correctamente el payload.

Serializar body

```
init.body = JSON.stringify(body);
```

Convierte el objeto JavaScript en texto JSON.

Ejemplo:

```
{ status: "ack" }
```

se convierte en:

```
{"status":"ack"}
```

Ejecutar fetch

```
const res = await fetch(url.toString(), init);
```

Realiza la petición HTTP.

Desglose:

```
url.toString()
```

Convierte el objeto URL en texto.

```
init
```

Configuración de la petición.

```
await
```

Espera a que la petición termine.

El resultado se guarda en:

```
res
```

que representa la respuesta HTTP.

Leer respuesta como texto

```
const text = await res.text();
```

Lee el cuerpo de la respuesta como texto.

Aunque se espere JSON, primero se lee como texto para poder manejar también respuestas vacías o errores no JSON.

Inicializar `data`

```
let data = null;
```

Crea una variable `data` inicialmente nula.

Aquí se guardará el JSON parseado si existe.

Intentar parsear JSON

```
try { data = text ? JSON.parse(text) : null; } catch { /* noop */ }
```

Si `text` tiene contenido, intenta convertirlo a JSON.

Desglose:

```
text ? JSON.parse(text) : null
```

Si hay texto, parsea JSON.

Si no hay texto, deja `data` como `null`.

El `catch` evita que la función falle si la respuesta no es JSON válido.

El comentario:

```
/* noop */
```

significa “no operation”.

Es decir, no se hace nada si falla el parseo.

Comprobar respuesta no OK

```
if (!res.ok) {
```

`res.ok` es `true` cuando el código HTTP está entre 200 y 299.

Si no está en ese rango, se considera error.

Ejemplos:

```
404  
409  
422  
500
```

Obtener detalle del error

```
const detail = data?.detail ?? text ?? `HTTP ${res.status}`;
```

Construye un mensaje de error.

Desglose:

```
data?.detail
```

Intenta obtener `detail` del JSON devuelto por FastAPI.

El operador `?.` evita error si `data` es `null`.

```
?? text
```

Si no hay `data.detail`, usa el texto de respuesta.

```
?? `HTTP ${res.status}`
```

Si tampoco hay texto, usa un mensaje genérico con el código HTTP.

Ejemplo:

```
{
  "detail": "Alert not found"
}
```

se convierte en:

```
Alert not found
```

Lanzar error

```
throw new Error(detail);
```

Lanza un error JavaScript.

Esto permite que las funciones que llaman a `apiFetch()` puedan capturarlo con `try/catch`.

Ejemplo en `alerts.js`:

```
catch (e) {
  show(err, `Error cargando alertas: ${e.message}`);
}
```

Retornar datos

```
return data;
```

Si la respuesta fue correcta, devuelve los datos parseados.

Ejemplo:

```
const data = await apiFetch("/alerts");
```

`data` contendrá la lista de alertas.

Función `getQueryParam`

```
function getQueryParam(name) {
  const u = new URL(window.location.href);
  return u.searchParams.get(name);
}
```

Lee un parámetro de la URL actual.

Se usa principalmente en `alert_detail.js` para obtener el ID de la alerta.

Crear URL actual

```
const u = new URL(window.location.href);
```

Crea un objeto `URL` a partir de la URL actual del navegador.

Ejemplo:

```
http://localhost:5173/alert.html?id=1
```

Obtener parámetro

```
return u.searchParams.get(name);
```

Devuelve el valor del parámetro indicado.

Ejemplo:

```
getQueryParam("id")
```

en esta URL:

```
alert.html?id=1
```

devuelve:

```
1
```

Si el parámetro no existe, devuelve `null`.

Función `setQueryParams`

```
function setQueryParams(params) {
```

Actualiza los parámetros de la URL actual.

Se usa en el listado para reflejar filtros y paginación en la barra de direcciones.

Crear URL actual

```
const u = new URL(window.location.href);
```

Crea un objeto `URL` usando la dirección actual.

Recorrer parámetros

```
Object.entries(params).forEach(([k, v]) => {
```

Recorre cada par clave-valor del objeto `params`.

Ejemplo:

```
{
  limit: 50,
  offset: 0,
  status: "open"
}
```

Eliminar parámetros vacíos

```
if (v === null || v === undefined || v === "") u.searchParams.delete(k);
```

Si el valor está vacío, elimina ese parámetro de la URL.

Esto evita dejar filtros vacíos en la barra de direcciones.

Ejemplo:

```
status=
```

se elimina.

Actualizar parámetros con valor

```
else u.searchParams.set(k, String(v));
```

Si el valor existe, lo escribe en la URL.

Ejemplo:

```
u.searchParams.set("status", "open")
```

Resultado:

```
?status=open
```

Reemplazar URL sin recargar

```
window.history.replaceState({}, "", u.toString());
```

Actualiza la URL visible del navegador sin recargar la página.

Esto es importante.

Permite que los filtros aparezcan en la URL, pero sin hacer una navegación completa.

Desglose:

```
window.history.replaceState
```

Modifica la entrada actual del historial.

```
{}
```

Estado asociado. Aquí se deja vacío.

```
""
```

Título. Normalmente no se usa.

```
u.toString()
```

Nueva URL.

Resultado final del archivo

Después de cargar `app.js`, quedan disponibles funciones globales para el resto del frontend:

```
API_BASE
qs
show
hide
fmtDate
statusBadgeClass
apiFetch
getQueryParam
setQueryParams
```

Estas funciones son utilizadas por:

```
alerts.js
alert_detail.js
```

Sin este archivo, las páginas no podrían:

- seleccionar elementos fácilmente
- mostrar mensajes de error
- ocultar mensajes
- formatear fechas
- llamar a la API de forma centralizada
- leer parámetros de URL
- modificar filtros en la URL

7 Relación con el flujo técnico del laboratorio

`app.js` no representa una pantalla concreta.

Representa la capa de apoyo del frontend.

La relación es:

```
index.html
  ↓
carga app.js
  ↓
carga alerts.js
  ↓
alerts.js usa apiFetch("/alerts")

alert.html
  ↓
carga app.js
  ↓
carga alert_detail.js
  ↓
alert_detail.js usa apiFetch("/alerts/{id}")
```

Flujo de llamada API:

```
JavaScript
  ↓
apiFetch()
  ↓
fetch()
  ↓
FastAPI
  ↓
respuesta JSON
  ↓
renderizado en HTML
```

8 Errores típicos o puntos importantes

API_BASE debe coincidir con el backend

Si el backend no está en:

```
http://localhost:8000
```

las llamadas fallarán.

Si cambia el puerto, hay que modificar:

```
const API_BASE = "http://localhost:8000";
```

app.js debe cargarse antes que otros scripts

En index.html :

```
<script src="./assets/app.js"></script>
<script src="./assets/alerts.js"></script>
```

En alert.html :

```
<script src="./assets/app.js"></script>
<script src="./assets/alert_detail.js"></script>
```

Este orden es necesario porque alerts.js y alert_detail.js usan funciones definidas en app.js .

apiFetch ignora query params vacíos

Esto es correcto porque evita enviar filtros sin valor.

Por ejemplo, si status está vacío, no se añade a la URL.

apiFetch convierte errores HTTP en errores JavaScript

Si FastAPI devuelve un error, apiFetch lanza:

```
throw new Error(detail);
```

Luego la pantalla lo muestra en errorBox .

fmtDate no valida si la fecha es inválida

La función intenta crear un Date , pero new Date(valor) puede producir Invalid Date sin lanzar excepción.

Para este MVP es suficiente, pero una versión más robusta podría comprobar:

```
isNaN(d.getTime())
```

statusBadgeClass está preparado para evolucionar

Ahora devuelve siempre:

```
badge
```

Pero podría ampliarse para devolver clases diferentes:

```
badge open
badge ack
badge closed
```

setQueryParams no recarga la página

Usa:

```
replaceState()
```

Por eso después de actualizar la URL normalmente se llama manualmente a la función que recarga datos.

En `alerts.js`, después de `syncUrl()` se llama a:

```
loadAlerts()
```

9 Comandos útiles relacionados

Servir frontend:

```
cd ~/siem-lab/frontend
python3 -m http.server 5173
```

Abrir página principal:

```
http://localhost:5173/index.html
```

Abrir detalle:

```
http://localhost:5173/alert.html?id=1
```

Probar backend usado por `apiFetch`:

```
curl http://localhost:8000/alerts
```

Probar actualización de estado usada por `apiFetch`:

```
curl -X PATCH http://localhost:8000/alerts/1 \
-H "Content-Type: application/json" \
-d '{
  "status": "ack"
}'
```

Abrir consola del navegador:

```
F12 → Console
```

Comprobar errores típicos:

```
Failed to fetch
CORS error
404 Alert not found
500 Internal Server Error
```